

EVMC talk notes

This is the result of running an interpreter written in EVMC.

It can do general-purpose computation; in this case I gave it a script implementing the factorial function with the argument 3. As you can see, it returns 6 - it works!

In the rest of the talk, I'm going to explain why that's really cool and important for Ethereum. But first, some background.

A blockchain is a decentralized database that anyone can access and no one can shut down.

The Ethereum blockchain's state is a mapping from addresses to accounts.

For users, the address is a public key and the account is an amount of money; they can transfer money to other accounts by sending them call transactions.

But there's another kind of account: smart contracts!

External users can do two things on the blockchain: send a call transaction or a create transaction.

A create transaction uploads a new smart contract at a pseudorandom address.

A call transaction sends a bytestring message and an amount of the native currency ether to the recipient. When a call is sent to an EOA (user), the only effect is to transfer ether.

But when a smart contract is called, its stored program gets run.

That program can read calldata, update its storage, call and create other contracts, and return a message. Now, how does that work?

When called, smart contracts run on the EVM (Ethereum Virtual Machine). It's a stack machine that gives the program access to a number of different addressable regions, each with their own set of instructions for accessing them.

Calldata contains the message passed to the contract; it's a read-only bytestring.

Memory is cheap volatile storage that's cleared when the call ends.

To remember things between calls, contracts can read and write their own much more expensive persistent storage.

To call and create other contracts, the contract passes a slice of memory using the call and create instructions respectively.

To end the call, the contract either returns or reverts (which undoes any state changes).

To prevent DoS attacks on the chain, all computational resources are metered via a unit of account called gas.

The amount of gas that can be used per block of transactions and the rate of block creation is limited, so users pay a fee proportional to the gas usage of their transactions. Ethereum is an extraordinarily expensive compute environment; a million additions will cost you about 70 cents.

Sending calldata in a transaction and executing EVM instructions both cost gas: uploading a byte of calldata costs 16 gas, while typical arithmetic instructions and memory access cost around 3 gas.

Operations which modify persistent state are much more expensive: calling another contract within a call costs 2600 gas (or about 900 adds), and writing a word to storage or creating a contract is even more costly.

In addition, there's a fixed cost of 21,000 gas paid per transaction sent before it even starts executing.

Those 4 operations are the biggest numbers in the cost model and account for the lion's share of transaction costs; they should be avoided as much as possible.

So why do people even use this ludicrously expensive virtual computer?

It's because smart contracts - programs that have no owner and can't be tampered with or shut down - are a powerful primitive for cooperation between mutually distrustful users.

Smart contracts are used to implement financial products such as new currencies, exchanges for trading them and even financial derivatives such as options and futures - all without the need for a trusted middleman.

Nowadays even assets from the traditional financial system are being moved on-chain as institutions have become more interested in the unique advantages of blockchains.

Hundreds of billions of dollars are already being managed by smart contracts, and that number is set to increase.

So, smart contracts are increasingly relevant in the financial system, but they're computationally limited, expensive to run and bugs cost millions because the contracts directly and irreversibly move money.

Surely EVM contracts must be implemented in very secure, very fast languages right? Well, not so much...

Let's talk about how they're written today.

Here's an excerpt of a contract written in Solidity, the most popular EVM language.

Solidity programs define an object with hidden state variables and public methods.

To implement a new currency, you could have a mapping as the state and send as a method, for example.

Send checks the sender has enough tokens and reverts if not; otherwise it decrements the sender's balance and increments the recipient's.
This is the standard way transferable assets are implemented on-chain.

This Solidity program is translated to its own EVM contract, which interprets its calldata as a method call and keeps its state in storage.
So deploying the program becomes a CREATE txn/ixn and each method call becomes a CALL txn/ixn which in turn triggers SSTORE operations.
The problem with that is that those are the most expensive operations in the cost model!
This expensive programming model is called contract-orientation and Solidity makes it mandatory.

But it's not just expensive: the model is also difficult to secure.
On the EVM, the contract is the trust boundary; outside it you can make no assumptions about behavior.
Because each Solidity program lives in its own contract, it must communicate across that boundary each method call. But anything could be out there!
All contracts are also public.
Contracts must assume anyone can call them... and when they call someone else the callee can behave arbitrarily - for example, it can call them back.
Failing to account for that led to the infamous DAO hack in 2016, where \$150M were lost.
The model is also problematic for stateful protocols, because each step in the protocol is translated to a method, which is then always available and can be called in the wrong order by attackers.

But there's a better way, through the power of PL!
You can think of Ethereum as an operating system and contracts as analogous to OS threads: the EVM gives them tamper resistance via isolation, but creating and context switching between them is expensive.
So could one make anything analogous to green threads? Yes you can!

By hosting several user programs in an interpreter EVM contract that guarantees safety properties, users can make use of those guarantees to safely share costs.
That's not just an academic idea, there are real-world examples of this: Batches let you share the fixed txn cost, for example.
A contract that handles multiple method calls in a loop can also share the cost of calling it.
A more complex interpreter could avoid CALL, CREATE and SSTORE entirely!
But existing languages are not well-suited to developing it.

When it comes to EVM languages, we're stuck between a rock and a hard place.

Sol locks you out of the machine and into the contract-oriented framework. You **can** write interpreters, but it's ill-suited for it due to its limited data structures, type system and control structures.

Solidity has enums, structs and arrays, but no algebraic data types.

That means instructions must either be split into arrays of opcode and immediate argument (with a PC for each), or you must pass a single array of structs emulating ADTs where most fields are unused.

Furthermore, Solidity has no switch or case construct; you must rely on if-clauses to match on opcodes.

Low-level languages let you do whatever you want, but don't help you be confident your very complex low-level program is correct - in particular, they have no type system and every value is treated as an EVM word.

We need a language between those extremes: one that gives you control of the machine and nice abstractions at the same time. We need a systems language!

Before we move on, I want to be clear about this: the goal is not a language that is X% faster than Solidity, but rather one that can express programs Solidity can't and that are too complex to develop in assembly.

EVMC

EVMC is a first step in that direction.

Here's an example contract written in it.

The program reads the first 10 bytes from calldata and returns their sum to the caller.

Already there are two big differences from Solidity:

First of all, we have a main function.

EVMC has no mandatory method dispatch; instead it has a single entrypoint, main!

That saves a little gas, but you could also just have a single method in Solidity. What's the big deal?

The real benefit is that it doesn't force you to declare the type of the calldata and parse it all at once. Note main just takes unit, so how can it parse anything from calldata? Using the second big difference:

EVMC has pointers!

You can coerce numbers to pointers and then give calldata whatever layout you like.

I didn't invent pointers ofc, but they're great - they give you complete control of everything but the stack, much more ergonomically than with the raw opcodes. Writing a value smaller than an EVM machine word (32B) takes multiple instructions, so doing it manually in assembly is a pain.

That example isn't vaporware, by the way: it actually runs!

In EVMC, the programmer is in complete control of the memory (modulo scratch). Often you want to allocate things; then you use `alloc` and its variants.

Note the Haskell-like type signature. The lowercase names are type variables. Unlike Solidity, EVMC functions are polymorphic, allowing for generic functions and abstractions to be defined once and then reused! `allocValue` here is an example of that.

It allocates space for one `a` using `alloc`, writes the `a` to that space and returns the pointer to it.

But how does `alloc` know how much space to allocate?

EVMC also has ad hoc

polymorphism; `alloc` uses the `sizeof primfun` (which takes a zero-size proxy value `P` just used to pass the type) to get the size of `a`.

Unlike Haskell, EVMC uses C++-style template polymorphism rather than dict passing.

That's to ensure it has no runtime overhead (which we can't afford).

A consequence of that is that polymorphic recursion where you call a function with a potentially infinite number of types is impossible.

If the number is bounded, it works fine though.

There are no class constraints, instead I just resolve instances after type inference. That may fail, but it makes for a lot less groveling before the type checker to prove instances can always resolve.

That lets me define recursive functions on tuples, which is very useful for encoding them to Solidity's method call format.

Looking at the type signature, you see `Ptr` takes two params, `r` and `a`. What's `r`? It's not a type - it's an EVM region!

I can represent that without any dedicated primitives because I have a kind system.

The user can declare new kinds such as `Region` and add new members to it using kind signatures.

These are (most of) the kinds provided by the `stdlib`.

`Type` is the kind of all types; all expressions have a type, and the type's kind is `Type`.

`Kind` has no kind, it's the root of the hierarchy; this isn't Agda.

`Nat` is a type-level natural number, and it's used to parameterize integers and arrays with their length.

The integer type constructor `Int` is also parameterized by its signedness, i.e. whether it's signed or unsigned.

This allows functions to be polymorphic in region and signedness as well as type.

Now, EVMC is a bit of a misnomer.

Beyond having Haskell-like type inference, I'm not limited to C-like enums, structs and pointers.

I also have algebraic datatypes! Here are some familiar Haskell types declared in the EVMC standard library.

All types are declared with a data decl, including Ptr and Int.

Ptr simply wraps a 16-bit uint, but

Int needs to have a primitive repr defined by the compiler.

So what can you do with this?

For one, I've implemented the Solidity ABI as a library, so

you can easily call Solidity methods and expose when you want to - without being forced to always do so.

That's an example of the design philosophy I borrowed from Haskell: by making the language minimal but composable, I can move features out of the core language and into libraries. That simplifies the core language and makes it more expressive.

Interface here is an example of how libraries can use the kind system.

It's a new kind denoting the exposed methods of a Solidity

contract; Solidity method tags and contract addresses are associated with their interface.

Look at `abi_call_method`'s signature: you see a contract with a given interface can only be called with methods that belong to that interface.

`abi_handle_method` lets you expose methods; it checks if the calldata has the given method tag and if so parses the rest as an a according to the Solidity ABI, gets a b from the handler function given to it, and returns the ABI encoding b to the caller.

You can also do cool functional stuff!

Closures pair a function pointer with a closed state, letting you create and compose new functions at runtime, rather than just having a fixed number of function pointers.

Closures let you obscure the internal representation of an API, for example a pointer to an a that a function should only write to can be replaced with a closure that takes the a as an argument and returns unit.

That lets you make functions more generic and composable, because they become agnostic to how the APIs they use are represented under the hood.

I use closures to define iterators, letting you fold over different container types using the same function.

Note how polymorphism allows me to define the Closure interface once and then use it to create and call closures of any type.

Polymorphism is essential to making new abstractions such as Closure reusable.

If you want to encapsulate data with several methods, you can use an object instead. Objects pair closed state with a method table pointer. I support inheritance for them, so you can automatically coerce an object to any of its superclasses (a superclass has a prefix of the subclasses' methods).

I also have thunks, but you probably shouldn't use them they're not terribly efficient. However, they can be used to represent infinite data structures and consume them incrementally.

Here's something much trickier: transparent coroutines.

Any EVMC function can be wrapped in a coroutine, making it concurrent. That's in contrast to JavaScript for example, where async functions are a different type from normal ones, preventing you from using preexisting HOFs such as map and fold in concurrent code.

This datatype is the coroutine's header.

A persisted stack is stored after the header in memory; when you resume a coroutine it fetches the stack from memory until it encounters the sentinel value in the header.

On yield, the stack is flushed to memory until the &coReturn pointer is encountered. If a value is coincidentally one of the sentinels, it'll break.

This requires dynamic stack access. EVMC can't do it directly because it has structured stack access, so I needed to use assembly.

I don't have any native support for inline asm, but fortunately I have enough control over datatype repr that I could implement it as a library!

Assembly sections are defined using code globals, immutable constants stored in code.

By coercing the address of a code global containing a struct of assembly instructions to a function, I can escape EVMC's static stack access.

Now for the crown jewel: the interpreter smart contract that I ran at the start of the talk.

Interpreter state is easy to define: it's just a struct containing program counter and registers. You could do that in Solidity as well.

Here's where EVMC really shines: I can define instructions as a boxed algebraic datatype! That means it's a pointer to one of the constructors under the hood; I can change which region I interpret instructions from by changing the r parameter of the boxed datatype.

That could be used to interpret from memory if you want to dynamically load new code, for example.

Now look how elegant ADTs make interpretation!

The dispatch loop is just a repeated case on the next instruction; there's a

Return instruction for exiting the loop, not pictured here.

The PC is coerced to an Instr, which is represented as a pointer so that just works.

LLM is a low-level machine that can perform arithmetic, control flow and arbitrarily modify memory. It doesn't provide any safety properties that would let multiple user programs safely share the same contract.

My goal is to write an interpreter that implements a runtime for a higher-level functional language with objects: closures, ADTs etc.

That would let users create and interact with VSCs which keep their state and code persistently stored in the interpreter.

VSCs are to EVM contracts what green threads are to OS threads.

That would let them share the fixed txn cost as in a batcher, but also eliminate the need for EVM CALL and CREATE operations; interaction could then be in memory which is cheap.

But contracts need to be stateful.

Surely updating these VSC's state requires SLOAD/SSTORE, right?

Yes, but by storing them all in a Merkle tree (a tree of hashes), you can just store the Merkle root and thus eliminate all but one SSTORE when modifying any number of VSCs.

In doing so, I will have eliminated all the big costs in the EVM (which account for the majority of the gas cost), trading them for cheaper volatile compute... and at the same time, give users a new, high-level programming model that can safely break the isolation of on-chain programs and let them trust each other!

For example, when a function receives a user-defined ADT and checks it's of type Prime, it'll know that Prime's invariants hold!

It's not clear what the interp overhead will be, but an LLM instr is currently 150 gas. More EVMC opts such as TCO and aggressive inlining should reduce that; there's still plenty of low-hanging fruit in the optimizer!

Since the instructions will be larger (e.g. calling a closure vs just loading to a register), the dispatch cost could be amortized better than in LLM. Overall, I'm hopeful that this could produce massive gas cost savings.

On the EVMC side, there's still more work to be done before it's ready for production.

For example, the type error messages are just terrible right now.

Furthermore, the compiler needs extensive testing before it's safe to use in production.

But I've validated that it can be used for its intended purpose.

So, that's EVMC and what I plan to do with it. Thank you for listening!